Assignment 2

Accompanying this assignment you will find an archive file `gtiit-sudoku-v2.zip`. The zip contains a directory `gtiit_sudoku_v2`, with a BlueJ Java project for the GTIIT Sudoku application. This is an extended version of the GTIIT Sudoku application version 1, of the first assignment of Introduction to Systems Programming.

Project *GTIIT Sudoku v2* is, as in the case of the first assignment, a text-based implementation of the Sudoku game. It consists of four classes:

- `SudokuCell`: it is the simplest of the classes. It models a cell of the Sudoku, with its possible states: fixed or not fixed, and its contents (value), with zero representing the cell being empty.
- `SudokuBoard`: it models the board through a list of rows, each row being a list of SudokuCell objects. It contains methods to create a board from a file, setting a value in a cell, checking if the board is solved, etc. This class has some extended and improved behavior: different implementations of `toString()`, and the possibility of *solving* a Sudoku board.
- `SudokuInputReader`: it implements the reading of input commands from the user.
- `SudokuMain`: the main class of the game. Its constructor creates a random Sudoku board (from a list of predefined ones), initializes the input reader object and game state, and enables initiating the game by calling method `play()`. The player can interact with the game in the same way as in version 1, with one additional action: `'s'` instructs the game to *solve* the puzzle.

Your task consists of various implementation stages.

Stage 1. The new project does not have your implementations in version 1. Incorporate your implementations of assignment 1, and make sure the project works as expected. Since the implementation of puzzle solving is not yet ready in this stage, make sure you interact with the game solely by using the commands of assignment 1 (filling the board, or quitting).

Additionally, we encourage you to write tests, especially for class `SudokuBoard`. These will be very important for later stages.

Make sure you save this first stage separately in a zip file (which has to be part of the final submission), before proceeding to the other stages. Include the test and test classes, if you have written them. You may name your archive `gtiit_sudoku_v2.1.zip`. Alternatively, if you are using Git (preferred option, as teams are easier to define), please commit a specific version for stage 1; you may indicate it is stage 1 solution by putting "stage 1 solution" in the comment of the commit, or creating a tagged commit with tag "stage1".

Stage 2. This stage builds upon the previous state, and will involve an important change in data representation. Instead of using a list of lists for representing the board contents, we will change this representation to an array of integers:

```
private SudokuCell[][] board;
```

Perform this change in data representation, and update all the methods of class `SudokuBoard` to meet this change. Make sure your implementation works as expected, as you did in stage 1, i.e., still by interacting with the game solely by using the commands of assignment 1 (filling the board, or quitting). The tests you wrote for stage 1 should all pass at this stage, and can have an important impact in guaranteeing your improved implementation preserves the behavior of the implementation of stage 1.

Again, you may write additional tests, if you find them useful.

Make sure you save this second stage separately in a zip file (which has to be part of the final submission), before proceeding to the other stages. Include the test and test classes, if you have written them. You may name your archive `gtiit_sudoku_v2.2.zip`. If you are using Git, the preferred option, please commit a specific version for stage 2; you may indicate it is stage 2 solution by putting "stage 2 solution" in the comment of the commit, or creating a tagged commit with tag "stage2".

Stage 3. This final stage builds upon the previous one, and will involve the implementation of Sudoku solving, using depth-first search. The methods you need to implement are the following, from class `SudokuBoard`:

- `validValueInCell(row, col)`. It checks if the value stored in the cell with coordinates (`row`, `col`) is valid, with respect to the Sudoku rules (non-conflicting with other values in the row, column, or box). Check if you implemented this behavior before. You may implement this method simply as a call to an already implemented method in stages 1 or 2.
- `SudokuBoard(SudokuBoard board)`. This is a *copying* or *cloning* constructor. It will be very important for the implementation of puzzle solving. This method must initialize the new `SudokuBoard` object by copying the data from the parameter `board`.
- `solve()`. It attempts to solve the Sudoku board, by performing a depth-first search exploration of all possible fillings of the unset cells of the board.

There are a couple of provided tests for this stage. You may write additional tests, if you find them useful.

Make sure you save this third stage separately in a zip file (which has to be part of the final submission). Include the tests and test classes, if you have written any. You may name your archive `gtiit_sudoku_v2.3.zip`. If you are using Git, the preferred option, please commit a specific version for stage 3; you may indicate it is stage 3 solution by putting "stage 3 solution" in the comment of the commit, or creating a tagged commit with tag "stage3".

In any case, if you need to update your solution, simply do so with similar tag or commit comment. The one taken into account for grading will be the latest with the corresponding commit or tag.

You may decide to add auxiliary methods as private, or implement the required functionalities within the provided method bodies. You are not allowed to change the APIs of the classes, i.e., not change method names, number of parameters or their types. You must not modify other existing methods in the implementation. We recommend to first study the project to understand how it is organized and how the funcitonalities are distributed in the four different classes, before starting with the implementation.

This assignment must be solved in groups of two. You must submit the solution through moodle, as a zip archive containing the zip files of all three different stages (the whole project at each stage). The correctness of your solution is important, as so is the quality of the code (clarity, readability, structure, etc.).